

# Summary of “Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters”

Anirudh Krishnan (anikrish), Evan Zimmerman (evanzimm), Naveen Unnikrishnan (naveenu), Pranavkumar Mallela (pmallela)

## Problem and Motivation

A lot of the performance growth in LLMs can be attributed to the massive scaling of their model parameters. However, Snell et al. try to answer the underexplored question: can more compute at test-time improve performance on a challenging prompt? This idea has similarities with our experience as a human: we tend to think longer on difficult problems to be able to solve them more effectively. The authors believe answering this question is a critical step towards “building generally self-improving agents that can operate on open-ended natural language”. Unraveling the scaling laws of test-time compute can not only help smaller models with such resources outperform larger models, but also enable researchers to understand and take advantage of the tradeoff between pre-training and test-time compute.

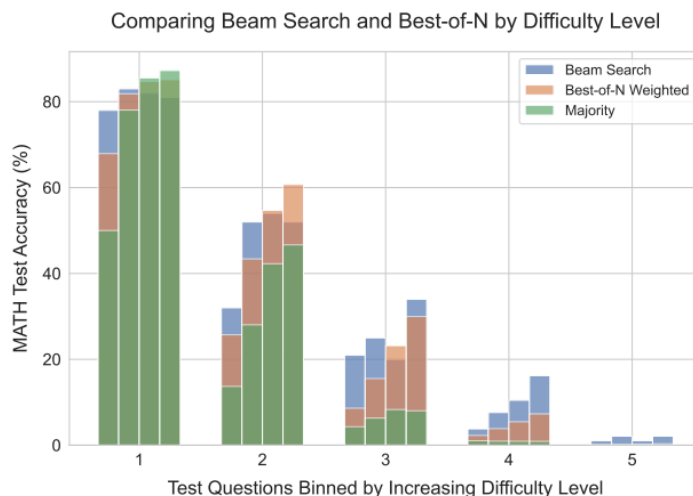
## Related Work

Builds on two main lines of research - (1) searching against dense, process-based verifier reward models and (2) updating the model’s distribution over a response adaptively, given the prompt at test time.

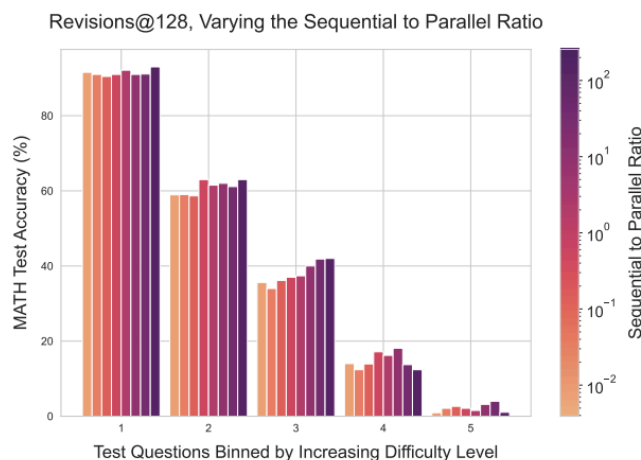
- **Best-of-N-Sampling** : The baseline for using extra compute in generating N independent out-puts and picking the best one via a verifier.
- **Process-based reward models (PRMs)**: Reasonable method to guide models toward better approaches for reasoning tasks but faces challenges with scalability, annotation, and reward hacking
- **Beam and Lookahead Search**: Search methods expand solutions step-by-step using PRM scores.
- **Revision based Models**: Models finetuned for revising iteratively with a focus to improve answers and enable compute-efficient performance gains.

## Solution Overview

Snell et al. utilize two main approaches to scale test-time compute and observe model performance. First, they search against dense process-based verifier reward models (PRM). Traditionally, ordinary reward models have been used to score the outputs of a model, which only score the final output. The PRM allows the authors to score every step in the model's chain of thought while arriving at the final output. Specifically, they employ search algorithms, Beam Search and Lookahead Search, to traverse the step-by-step prediction space and select the best answer.



Second, they dynamically modify the distribution of the model's response, given the prompt at test-time. The paper refers to models with such capabilities as revision models. Instead of naively prompting the model to revise its answer, the authors finetune a model on generated revision data. The data was generated by pairing up each correct answer with up to four incorrect answers and employing edit distance to calculate the correlation between them. There are two modes of revision discussed by the authors: sequential conditioning and parallel sampling. Sequential lets the model condition on its previous answers to improve the current answer. Parallel generates N outputs independently in parallel.



Finally, the authors propose compute-optimal strategies for both approaches. For PRMs (verifiers), beam-search is more effective on harder questions and at lower compute budgets, whereas best-of-N is more effective on easier questions and at higher budgets. For the revision model, easier questions benefit from purely sequential TTC, whereas harder questions often perform best with some ideal ratio of sequential to parallel compute.

## Limitations

- Estimating difficulty causes extra TTC and the current experiments do not account for it , as it is assumed to be much less than actual experiment cost.
- At high generation budget, beam and lookahead search can overfit PRM, leading to degenerate responses.
- For the hardest MATH problems , adding additional TTC shows minimal benefit , they suggest more pre-training in that case.

## Future Research Directions

- **Further improving test-time compute scaling:** Explore richer combinations of test-time methods, such as PRM tree-search with revision models or critique-and-revise approaches, to further improve scaling.
- **Assessing question difficulty quickly:** The current approach uses a compute-heavy procedure to bin questions by difficulty before allocating compute. Future work should explore cheaper alternatives such as pretrained difficulty predictors or dynamic strategies that switch between assessing difficulty and solving the question in real time.
- **Interleaving test-time and training-time compute:** Test-time compute currently improves performance only at inference. A natural next step is to distill these improved outputs back into the base model, creating an iterative self-improvement loop that blends training-time and inference-time computation.

## Summary of Class Discussion

*Q: Does sequential revision mean the model changes its output one token at a time?*

A: No, we revise a small output at a time (a bunch of tokens or the whole answer). Sequential revision is more like how a person answers: writing something, then going back to revise portions as needed. The granularity varies depending on the implementation. This approach likely requires more memory than parallel revision because of how edits accumulate, but the paper should be referenced for specifics. Importantly, the paper allocates equal test-time compute (TTC) to both sequential and parallel approaches for fair comparison.

*Q: Are these methods describing models or systems?*

A: They are closer to systems than simple downloadable models. For instance, PRM (Process Reward Model) is a separate verifier model, while Proposer/Revisor is structured more like a single model. This means extra scaffolding is often needed around the base model for these methods to work. The professor compared this difference to a “take-home exam vs in-person exam” — one needs more external structure, while the other is self-contained.

*Q: How actionable is the third experiment (Pretraining vs Post-training)?*

A: The tradeoff can be actionable. For example, instead of training a very large model, one might use a smaller model combined with more TTC at inference. However, the actionability is limited because pretraining and TTC happen at different stages. To evaluate fairly, one would

need to consider the *end-to-end cost* over the model's lifetime: large model upfront vs. small model with repeated inference-time compute. Measuring FLOPs here is non-trivial, since it depends on how many inferences are run over the lifetime of the model. If accuracy differs, the comparison becomes even messier (a 3D tradeoff). In practice, only large organizations may be able to run such cradle-to-grave studies. In short, the FLOPs tradeoff is not as simple as they put it. They just ran it on a small dataset. We may see FLOPs for the small model cross that for a large model.

*Q: Can the verifier and proposer be combined into a single model?*

A: This was suggested as a possible future direction but was not studied in the current work. The professor noted that different methods imply different computational graphs, so it would be interesting to study how compute is distributed when combining them.

# Summary of “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning”

Anirudh Krishnan (anikrish), Evan Zimmerman (evanzimm), Naveen Unnikrishnan (naveenu), Pranavkumar Mallela (pmallela)

## Problem and Motivation

Recent large language models like OpenAI’s o1 have made breakthroughs in reasoning through the introduction of inference-time scaling by increasing the length of the Chain-of-Thought (CoT) reasoning process. However, there is still a lack of a general method for improving reasoning ability without heavy reliance on Supervised Fine-Tuning (SFT) on large, curated datasets. Prior approaches like process-based reward models, reinforcement learning (RL), and search algorithms such as Monte Carlo Tree Search and Beam Search have been unable to match o1’s reasoning performance.

The paper attempts to create a general method for enabling reasoning in LLMs and demonstrates that:

1. Pure rule-based RL can incentivize reasoning behavior in LLMs
2. A small post-training pipeline using cold-start data and multi-stage RL can generate a user-friendly and reliable reasoning model
3. The reasoning patterns of larger models can be distilled into smaller models, and this results in better performance compared to the reasoning patterns discovered in smaller models directly through RL

## Related Work

- **OpenAI o1:** Introduced inference-time scaling by lengthening chain-of-thought reasoning, boosting performance in reasoning tasks.
- **Process-based reward models (PRMs):** Reasonable method to guide models toward better approaches for reasoning tasks but faces challenges with scalability, annotation, and reward hacking.
- **Search-based approaches:** Methods like Monte Carlo Tree Search and Beam Search attempt systematic solution exploration but struggle with the vast search space of token generation.
- **Reinforcement Learning for Self-Correction:** Prior work introduced a multi-turn online RL method that allows a model to iteratively detect and fix its own mistakes using only self-generated correction traces. This RL approach overcame issues of distribution mismatch and behavior collapse that hinder supervised fine-tuning.

## Solution Overview

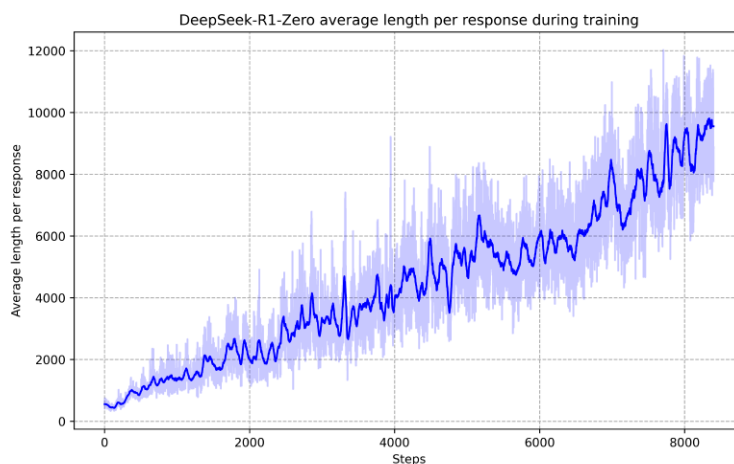
The paper describes the training of DeepSeek-R1-Zero and DeepSeek-R1, both using DeepSeek-V3 as a base model.

### DeepSeek-R1-Zero: RL on the Base Model

RL is performed using Group Relative Policy Optimization (GRPO) directly on the base model without any supervised data. The model is rewarded for both accuracy (is the response correct?) and formatting (is the thinking process enclosed by `<think>` and `</think>?`).

During the RL training process, a consistent improvement in the thinking time of the model is observed. The model naturally acquires the ability to solve increasingly complex reasoning problems by leveraging extended test-time compute. Complex behaviors such as reflection and exploration of alternative approaches spontaneously emerge during the training.

Another interesting observation is the emergence of “aha moments”. The model naturally learns to allocate more thinking time to a problem and re-evaluate its initial approach.



---

Question: If  $a > 1$ , then the sum of the real solutions of  $\sqrt{a - \sqrt{a+x}} = x$  is equal to

Response: `<think>`

To solve the equation  $\sqrt{a - \sqrt{a+x}} = x$ , let's start by squaring both ...

$$(\sqrt{a - \sqrt{a+x}})^2 = x^2 \implies a - \sqrt{a+x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a+x}} = x$$

First, let's square both sides:

$$a - \sqrt{a+x} = x^2 \implies \sqrt{a+x} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...

---

Table 3 | An interesting “aha moment” of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Though DeepSeek-R1-Zero exhibits strong reasoning capabilities, it has poor readability and mixes languages in its responses.

### DeepSeek-R1: RL with Cold Start

To address the limitation of DeepSeek-R1-Zero, DeepSeek-R1 is created where the training is augmented with cold start data. The new training pipeline has four stages.

#### Stage 1: Cold start

A small amount of chain-of-thought data is collected to finetune the base model before RL. It is found that this helps with both readability and model performance after RL.

## **Stage 2: Reasoning-Oriented RL**

The same RL training process is employed. However, to address language mixing, an additional language consistency reward is included. This results in slightly lower model performance, but improves readability and makes the model more accessible to human users.

## **Stage 3: Rejection Sampling and SFT**

After stage 2, curated reasoning prompts and rejection sampling is used to generate ~600k samples of reasoning data for further fine-tuning. For non-reasoning data, ~200k samples from DeepSeek-V3's SFT dataset is used.

The model is then finetuned using this curated dataset for two epochs.

## **Stage 4: Secondary RL**

Finally, an additional RL stage using a diverse variety of prompts and reward models helps improve the model's helpfulness and harmlessness.

## **Distillation into Small Models**

To equip more efficient smaller models with reasoning capabilities like DeepSeek-R1, open-source models like Qwen and Llama were fine-tuned using the 800k samples curated with DeepSeek-R1. The authors show that this straightforward distillation method significantly enhances the reasoning abilities of smaller models. On the other hand, they find that relying on large-scale RL for smaller models need not achieve the performance obtained through distillation.

## **Limitations**

For DeepSeek-R1-Zero specifically, poor readability and language mixing was a major limitation. DeepSeek-R1-Zero often produced answers that mixed languages (i.e. English and Chinese) and failed to output in markdown format. These limitations were addressed through incorporating cold start data into the RL process of DeepSeek-R1 as described in the solution overview section of this summary.

Because DeepSeek-R1 is optimized for zero-shot prompts (single question with the required output format and other necessary context), performance suffers in few-shot prompting scenarios. This is a limitation because users may often naturally utilize few-shot prompting. Performance degradation on few-shot prompting suggests that DeepSeek-R1 struggles with incorporating previous examples when trying to execute on a prompt. This may be because the RL process rewarded solving tasks directly over considering prior examples.

A key factor in the success of any RL process is the RL model's ability to repeatedly sample and evaluate trajectories. For long, complicated tasks like software engineering, where you have to compile and run/test programs, the process of sampling and evaluating trajectories can take a long time. Spending a long time on sampling and evaluating, was a bottleneck for DeepSeek's

RL training on complex, lengthy tasks because it hinders the efficiency of the training process and decreases training throughput.

## Future Research Directions

The authors identify a handful of research directions for DeepSeek-R1:

- **Utilizing Long Chain-of-thought** for tasks such as function calling, and multi-turn, complex role-playing to achieve parity with DeepSeek-V3 on general capability tasks
- **Effectively handling languages other than English and Chinese.** Since DeepSeek-R1 is optimized for English and Chinese, sometimes it will reason and respond in those languages even when the query is in a different language.
- **Improving prompt sensitivity.** Zero-shot prompting achieves optimal results, while few shot prompting usually performs worse. This leaves improving few-shot results as an area for future research.
- **Incorporating rejection sampling and/or asynchronous evaluations** to the RL process to improve performance on software engineering tasks because applying large-scale RL to engineering tasks was a challenge due limitations of the RL process.

## Summary of Class Discussion

*Q: Would the average length per response hit a ceiling during RL training or would average length per response keep increasing as training/accuracy increases?*

A: There is not necessarily a direct correlation between thinking length and model performance. Longer responses likely only help up until a point. Other work has shown that reasoning models can sometimes “overthink” on tasks, wasting time and generation (i.e. lots of redundant filler output), potentially causing worse performance. It is important to note that accuracy improves with more RL training steps. This is a direct correlation with RL training, and only a correlation with response length as a byproduct of longer RL training. There is no “proof” in the mathematical sense that response length is a contributing factor to accuracy. This is a case of correlation, but not necessarily causation.

*Q: Was there any ablation study mentioned with respect to the different components of the R1 training pipeline? For example, why do they specifically do 2 epochs of SFT?*

A: The R1 pipeline was roughly based on the V3 pipeline. They didn’t really mention any ablation studies in this paper. The reason behind 2 epoch SFT is likely that they did not want R1 to stray from V3 too much. As for the entire pipeline as a whole, the presenters emphasized that DeepSeek tried PRM and MonteCarlo tree search but that didn’t produce good results for their architecture.

*Q: Does R1 do any revisions/self improvement? Is it just a standalone model or a system?*

A: The short answer is that R1 is just a model. While it iterates on its thoughts during the generation process (chain-of-thought), there is no standalone verification component. R1 may



“restart” thinking if needed, but not in the sense that it is pruning solution branches during search (like the search algorithms in the Scaling LLM Test-Time Compute paper).

*Q: If you don't cold-start R1-Zero, how do you initiate RL without “superficial” pattern matching?*

A: Regardless of cold-start or not it is important to understand that the RL process is always going to do some version of “superficial” pattern matching (“superficial” in the sense that it will identify patterns on its own). R1(with cold start + SFT + RL) is still doing pattern matching; the cold-start just gives the RL process a better baseline for long CoT and output format (i.e. structured and human readable).

*Q: Do you think they attribute more significance to R1's “aha moment” than there really is?*

A: With R1, we give the model more time to think and catch itself. The significance of the “aha moment” is that we don't ever explicitly tell the model to correct itself (or find a better way to solve a problem), it does that on its own.